Apache Flink Cheat Sheet

Flink Checkpoint vs Savepoint

These serve related but distinct purposes in Flink's fault tolerance and state management story.

Checkpoints — Automatic Fault Tolerance

Checkpoints are Flink's **automatic, internal recovery mechanism**. The system manages them.

Purpose: Recover from unexpected failures (JVM crash, network partition, task failure)

Key characteristics:

- Triggered automatically by Flink at a configured interval
- Lifecycle is managed by Flink — expired checkpoints are deleted automatically
- Lightweight by design; optimized for speed over completeness
- Stored in a configured state backend (RocksDB on disk, or heap)
- Not portable across job topology changes (by default)
- Incremental checkpointing available (especially with RocksDB) — only delta is written

Config levers (Flink SQL / Table API):

```
SET 'execution.checkpointing.interval' = '60s';
SET 'execution.checkpointing.mode' = 'EXACTLY_ONCE';
SET 'execution.checkpointing.min-pause' = '10s';
SET 'state.backend' = 'rocksdb';
SET 'state.checkpoints.dir' = 's3://your-bucket/checkpoints';
```

On Confluent Cloud for Apache Flink (CCAF): Checkpointing is fully managed — you don't configure intervals or backends directly. Confluent handles it transparently per statement.

Savepoints — Operator-Triggered Snapshots

Savepoints are **manually triggered, portable state snapshots**. You own them.

Purpose: Planned operations — upgrades, topology changes, A/B migrations, backfills, cloning

Key characteristics:

- Triggered explicitly (`flink savepoint <job-id>`) or via REST API
- Persist indefinitely — never auto-deleted
- Stored in a canonical, stable format (not incremental)
- Portable across compatible job graph changes *if operator UIDs are stable*
- Can resume from a savepoint on a **modified job** (add/remove operators with caveats)
- More expensive to take than a checkpoint

Critical requirement — operator UIDs:

```
// Without stable UIDs, savepoint restore fails on redeployment
env.addSource(kafkaSource)
   .uid("kafka-source-v1")           // ← MUST be stable
   .keyBy(...)
   .process(new MyStatefulFn())
   .uid("my-stateful-processor-v1")  // ← MUST be stable
   .addSink(...)
   .uid("iceberg-sink-v1");
```

Without `.uid()`, Flink auto-generates UIDs from the job graph hash — any topology change breaks restore.

Decision Matrix

Concern	Checkpoint	Savepoint
Job crash recovery	✅ Primary use	❌ Not typical
Rolling upgrade / redeploy	❌	✅
Job topology change	❌ Usually broken	✅ With stable UIDs
Scheduled backup / archival	❌	✅
Auto-managed lifecycle	✅	❌ You manage it
Performance overhead	Low (incremental)	Higher (full snapshot)
Confluent Cloud support	Fully managed	via REST API

Confluent Cloud for Apache Flink (CCAF) Nuances

On CCAF, the savepoint story is surfaced through the Confluent Cloud REST API. A few things to know:

- **Statement-level recovery** is checkpoint-driven; Confluent retains recent checkpoints per statement
- **Savepoints** are accessible via the Confluent Cloud REST API — useful for migrating statements between compute pools or upgrading logic

---

Flink Window Types

Flink provides several window types to group infinite streams into finite, processable chunks. Here's a breakdown:

---

1. Time Windows

Tumbling Windows

- Fixed-size, non-overlapping, contiguous windows

- Every element belongs to exactly one window
- Example: "sales per hour" — each hour is a distinct window
- `TumblingEventTimeWindows.of(Time.hours(1))`

Sliding Windows

- Fixed-size but *overlapping* windows defined by window size + slide interval
- An element can belong to multiple windows
- Example: "avg over last 10 min, updated every 5 min"
- `SlidingEventTimeWindows.of(Time.minutes(10), Time.minutes(5))`

Session Windows

- Dynamic-size windows based on activity gaps (no fixed size)
- A new window starts when a gap of inactivity exceeds a threshold
- Great for user session analysis
- `EventTimeSessionWindows.withGap(Time.minutes(5))`

Global Windows

- A single, unbounded window for all elements with the same key
- Requires a custom **Trigger** to actually process anything (otherwise nothing fires)
- Used for fully custom windowing logic

---

2. Count Windows

- Based on number of elements, not time
- **Tumbling Count**: fires when N elements arrive — `countWindow(100)`
- **Sliding Count**: fires every N elements, keeping last M — `countWindow(100, 10)`

---

3. Time Semantics (applies to time-based windows)

Semantic	Clock Used	Notes
Event Time	Timestamp embedded in the event	Most accurate; requires watermarks
Processing Time	Wall clock of the operator	Simplest; non-deterministic
Ingestion Time	Timestamp when event enters Flink	Compromise between the two

Event time is almost always preferred in production because it gives deterministic, replayable results regardless of processing delays.

---

4. Advanced / Custom

- **Window Assigners**: You can implement `WindowAssigner` to create fully custom window shapes
- **Triggers**: Control *when* a window fires (e.g., early/late firings with `allowedLateness`)
- **Evictors**: Remove elements from a window before/after processing

---

Quick Decision Guide

```
Need fixed, non-overlapping buckets?     → Tumbling
Need overlapping / rolling averages?     → Sliding
Need activity-based grouping?            → Session
Need element count-based processing?     → Count
Need fully custom logic?                 → Global + custom Trigger
```

Flink SQL Topology

In Flink SQL, **topology** refers to the logical and physical execution graph that Flink builds to process your SQL queries as a streaming pipeline. It's essentially the DAG (Directed Acyclic Graph) of operators that transforms your input streams into output results.

Key Concepts

1. Query → Operator Graph When you submit a Flink SQL query, the planner translates it into a chain of operators:

```
Source → [Transform Operators] → Sink
```

Each SQL clause maps to one or more operators in the topology.

2. Logical Plan vs. Physical Plan

- **Logical Plan** — The abstract relational algebra tree (e.g., `TableScan → Filter → Project → Aggregate`)
- **Physical Plan** — The optimized execution plan after the Flink/Calcite optimizer applies rules (e.g., choosing hash join vs. sort-merge join, pushing down filters)
- **Execution Graph** — The actual parallelized runtime topology deployed to the cluster

---

Common Operator Types in a Flink SQL Topology

SQL Construct	Operator(s) in Topology
`FROM table`	**Source** (Kafka, filesystem, etc.)
`WHERE`	**Filter / Calc**
`SELECT`	**Project / Calc**
`JOIN`	**HashJoin, IntervalJoin, TemporalJoin, or LookupJoin**
`GROUP BY` + aggregation	**StreamGroupAggregate or GroupWindowAggregate**
`OVER` window	**OverAggregate**

SQL Construct	Operator(s) in Topology
`UNION ALL`	**Union**
`INSERT INTO`	**Sink**

Streaming-Specific Topology Considerations

Because Flink SQL runs on **unbounded streams**, the topology has important nuances vs. batch SQL:

- **Changelog Mode** — Operators produce `+I` (insert), `-U` (retract before), `+U` (update after), `-D` (delete) changelog records. This affects how downstream operators are wired.
- **Stateful Operators** — Aggregations, joins, and deduplication maintain keyed state. The topology includes state backends behind these operators.
- **Watermarks** — Event-time windowing inserts watermark tracking into the source and propagates it through the topology.
- **Mini-batch Optimization** — The optimizer may insert buffer operators to micro-batch records before stateful operators for throughput.

Visualizing the Topology

You can inspect the topology at multiple levels:

```sql
-- See the logical/physical plan
EXPLAIN SELECT ...;

-- See the full execution plan with parallelism, state info
EXPLAIN PLAN FOR SELECT ...;
```

In the **Flink Web UI**, the topology is rendered as a job graph showing:

- Each operator node
- Parallelism per operator
- Data exchange patterns (forward, hash, rebalance, broadcast)
- Backpressure indicators per edge

Example: Simple Aggregation Topology

```sql
INSERT INTO sink_table
SELECT user_id, COUNT(*) AS event_count
FROM kafka_source
WHERE event_type = 'click'
GROUP BY user_id;
```

The resulting topology looks like:

```
KafkaSource (parallelism=N)
    │  [forward]
    ▼
Calc (Filter: event_type='click', Project: user_id)
    │  [hash by user_id]
    ▼
StreamGroupAggregate (stateful, keyed by user_id)
    │  [forward]
    ▼
Sink (parallelism=N)
```

On Confluent Cloud for Apache Flink (CCAF)

In your context with **Confluent Cloud Flink**, the topology is managed by the Confluent runtime but you can still:

- Use `EXPLAIN` to inspect plans
- View job graphs in the Confluent Cloud UI
- Tune parallelism hints and table options to influence the physical topology
- Observe operator-level metrics (records in/out, state size, watermarks) per node

Understanding the topology is critical for diagnosing **backpressure**, **state bloat**, and **late event handling** issues in your streaming pipelines.

In Apache Flink, there are three notions of time:

1. Event Time The time embedded in the data itself — when the event actually occurred at the source. This is the most meaningful for business logic (e.g., a sensor reading timestamp, a transaction time). Event time processing uses **watermarks** to handle out-of-order and late-arriving events. Results are deterministic and reproducible regardless of when the data is processed.

2. Processing Time The wall-clock time of the machine executing the Flink operator at the moment it processes the record. It requires no coordination or watermarks, making it the lowest-latency and simplest option — but results are non-deterministic because they depend on when data arrives and how fast the system processes it.

3. Ingestion Time The time at which a record enters the Flink source operator. It's automatically assigned by the source and used as the event time from that point forward. It sits between event time and processing time: more consistent than processing time (no skew between operators), but less meaningful than true event time since it reflects arrival at Flink, not when the event actually happened.

Quick comparison:

	Event Time	Ingestion Time	Processing Time
Source of time	Data itself	Flink source	Flink operator

	Event Time	Ingestion Time	Processing Time
Deterministic?	✅ Yes	Partially	❌ No
Needs watermarks?	✅ Yes	Auto-assigned	❌ No
Handles late data?	✅ Yes	Limited	❌ No
Latency	Higher	Medium	Lowest

In practice, **event time** is by far the most commonly used in production pipelines — especially relevant for your streaming work on Confluent Cloud where out-of-order events are a real concern.

Apache Flink Watermarks

A **watermark** is Flink's mechanism for tracking progress in **event time**. It's a special record injected into the data stream that asserts:

> *"I am confident that no events with a timestamp earlier than **T** will arrive in this stream."*

How It Works

Watermarks flow through the stream alongside regular data records. When a watermark with timestamp **T** reaches a window or time-sensitive operator, Flink knows it can safely trigger computations for all events with `event_time ≤ T`.

```
Events:  [t=1] [t=4] [t=2] [t=7] [t=5] ~~~W(6)~~~> [t=9] [t=8]...
                                            ↑
                                Watermark T=6 triggers all
                                windows closing at or before t=6
```

Watermark Strategies

1. Monotonically Increasing (Zero Tolerance) Assumes events arrive in perfect order. Watermark = max event timestamp seen so far. No late events tolerated.

```
WatermarkStrategy.forMonotonousTimestamps()
```

2. Bounded Out-of-Orderness (Most Common) Assumes events can arrive late by at most a known duration. Watermark = max timestamp seen − max out-of-orderness.

```
WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(5))
```

3. Custom Watermark Generator Full control — useful for sparse streams, per-key watermarks, or external signals.

Idle Partitions Problem

A critical real-world issue: if one Kafka partition stops receiving events, its watermark **stops advancing**, which **stalls the entire pipeline** because Flink takes the **minimum watermark across all parallel subtasks**.

```
Partition 0: ~~~W(100)~~~W(105)~~~W(110)...   ✅ advancing
Partition 1: ~~~W(100)~~~ (silent)             ❌ stalled
                              ↑
               Global watermark stuck at W(100)
```

Fix: Mark idle sources so Flink excludes them from the minimum calculation:

```
WatermarkStrategy
    .<MyEvent>forBoundedOutOfOrderness(Duration.ofSeconds(5))
    .withIdleness(Duration.ofSeconds(30))
```

Late Events

Even with a well-tuned watermark, some events will arrive *after* their window has already fired. You have three options:

Strategy	Behavior
Drop (default)	Late events are discarded
Allowed Lateness	Window kept open for extra duration, re-fires on late arrivals
Side Output	Late events routed to a separate stream for auditing/reprocessing

Side outputs as an observability pattern for late events is something you've explored before — useful for monitoring watermark lag in production.

Watermarks in Confluent Cloud Flink

In Confluent Cloud's managed Flink (Flink SQL), watermarks are defined at the **table/source level** using the `WATERMARK FOR` clause in DDL:

```
CREATE TABLE orders (
    order_id    STRING,
    amount      DECIMAL,
```

```
    order_time  TIMESTAMP(3),
    WATERMARK FOR order_time AS order_time - INTERVAL '5' SECOND
) WITH (
    'connector' = 'confluent',
    'kafka.topic' = 'orders'
    ...
);
```

The watermark expression (`order_time - INTERVAL '5' SECOND`) defines your out-of-orderness tolerance directly in SQL.

Key Takeaway

Watermarks are the **heartbeat of event-time processing** — they let Flink balance **completeness** (waiting for late data) vs. **latency** (triggering results quickly). Tuning your watermark strategy is one of the most impactful performance and correctness decisions in any Flink pipeline.

Allowed Lateness in Apache Flink

Allowed Lateness is a mechanism that keeps a window's state alive for an additional period *after* the watermark has passed the window's end time — giving late-arriving events a second chance to be included and causing the window to **re-fire** with updated results.

The Problem It Solves

With a standard watermark strategy, once the watermark passes a window's end time, the window fires and its state is immediately **discarded**. Any event arriving after that is simply dropped.

```
Window: [0:00 → 0:10]
Watermark passes 0:10 → window FIRES → state DELETED
Late event arrives at t=0:08 → DROPPED ❌
```

Allowed Lateness says: *"Keep the window state alive a little longer just in case."*

How It Works

```
Window end:        t=10
Watermark hits t=10  →  Window FIRES (first/on-time result)
                                ↓
                    State kept alive for allowed lateness duration
                                ↓
Late event arrives t=8  →  Window RE-FIRES (updated result)
Late event arrives t=9  →  Window RE-FIRES again (updated result)
```

```
                                    ↓
Watermark hits t=10 + allowed lateness  →  State PURGED forever
```

So a window can fire **multiple times** — once on time, then again for each late event that arrives within the allowed lateness window.

DataStream API Example

```
DataStream<MyEvent> stream = ...;

stream
    .keyBy(MyEvent::getKey)
    .window(TumblingEventTimeWindows.of(Time.minutes(10)))
    .allowedLateness(Time.minutes(2))          // keep state 2 min past
watermark
    .sideOutputLateData(lateOutputTag)         // capture anything beyond
2 min
    .aggregate(new MyAggregator());
```

Flink SQL

In Flink SQL (including Confluent Cloud), allowed lateness is configured via table/query hints:

```
SELECT
    window_start,
    window_end,
    COUNT(*) AS event_count
FROM TABLE(
    TUMBLE(TABLE orders, DESCRIPTOR(order_time), INTERVAL '10' MINUTE)
)
GROUP BY window_start, window_end;
```

Set lateness at the source table level:

```
CREATE TABLE orders (
    order_id   STRING,
    amount     DECIMAL,
    order_time TIMESTAMP(3),
    WATERMARK FOR order_time AS order_time - INTERVAL '5' SECOND
) WITH (
    'changelog.mode' = 'retract',   -- needed for late updates
    ...
);
```

Late updates in SQL emit **retract + insert** pairs (changelog semantics) rather than simple re-fires.

Allowed Lateness vs. Watermark Slack vs. Side Output

These three are often confused but serve distinct purposes:

Mechanism	Purpose	Cost
Watermark slack (`BoundedOutOfOrderness`)	Delay watermark advancement to absorb out-of-order events *before* first fire	Adds latency to all windows
Allowed Lateness	Keep window state alive after first fire for late stragglers	Memory (state kept longer)
Side Output	Capture events that are truly too late (beyond allowed lateness)	Separate stream to handle/monitor

The **recommended production pattern** is to use all three together:

```
Watermark slack        →  handles most out-of-order events cleanly
+ Allowed Lateness     →  catches the occasional straggler, re-fires
result
+ Side Output          →  captures truly late data for auditing/alerting
```

Important Caveats

State cost — Windows with allowed lateness hold state longer. In RocksDB-backed stateful jobs (common in Confluent Cloud Flink), this means more state storage and potential pressure on CFU memory. Size your allowed lateness window conservatively.

Downstream idempotency — Since a window can fire multiple times, your downstream consumers (Kafka topics, Iceberg tables, etc.) must handle duplicate/updated results. This is especially relevant when sinking to Iceberg via Flink where upsert/merge semantics need to be accounted for.

Retract streams in SQL — Late updates in Flink SQL produce changelog messages. Your sink connector must support retraction (e.g., using `'changelog.mode' = 'retract'`) otherwise late updates will be silently dropped or cause errors.

Why Automated Side Output Reprocessing Is Problematic

Ordering guarantees break down When you pull late events from a side output stream and feed them back into processing, you've lost the original event time context relative to the main stream. The reprocessed events may land in a different window than intended, or trigger further late arrivals downstream — a cascading problem.

Complexity and operational burden You now have two streams to monitor, two sets of consumer lag to track, two failure modes to handle, and logic to ensure the reprocessed side stream doesn't itself produce

late events into *another* side output. It compounds quickly.

Downstream consistency is hard to guarantee If your sink is Iceberg, a Kafka topic, or a database, the re-fired window results from reprocessed late events can arrive out of order relative to what's already been written. Without careful upsert/merge semantics, you get corrupted or inconsistent state downstream.

Hidden infinite loop risk If the reprocessing pipeline has its own watermark strategy and lateness tolerance, a poorly tuned setup can create a feedback loop where late events keep cycling.

What Side Outputs Are Actually Good For

The more practical and operationally sound use of side outputs for late data is **observability and forensics**, not automation:

- **Monitor** how many late events are being dropped and at what lag
- **Alert** when late event volume exceeds a threshold (signals watermark misconfiguration)
- **Audit** late records to a dead-letter Kafka topic or S3/Iceberg for manual review
- **Tune** your watermark slack and allowed lateness based on real late-arrival patterns you observe

In other words — side outputs tell you *something is wrong* with your watermark strategy, rather than silently papering over it with automated reprocessing.

The Better Alternatives

If late event handling is a genuine business requirement (not just an occasional straggler), the more robust approaches are:

Approach	When to Use
Widen watermark slack	You have measurable, consistent out-of-orderness — just absorb it upfront
Increase allowed lateness	Occasional stragglers that must be counted, downstream supports retraction
Batch reconciliation	Periodically reprocess a time window from the raw Kafka topic or Iceberg table to correct results
Lambda/Kappa architecture	Separate speed layer (streaming) + accuracy layer (batch reprocessing from source of truth)

Batch reconciliation from Iceberg is probably the most defensible pattern — the raw events are already durably stored, so a scheduled Flink batch job or Flink SQL query over Iceberg can recompute any window correctly without the complexity of live side output reprocessing.

Apache Flink Job Hierarchy: Job → Operator → Task → Subtask

The Hierarchy

```
Job
 └── Operator(s)
      └── Task(s)  [one per operator chain / parallel slice]
           └── Subtask(s)  [parallel instances of a Task]
```

Operator

An **Operator** is a logical processing unit in your Flink job — it represents a single transformation step in your dataflow graph.

Examples:

- A `MAP` operator
- A `FILTER` operator
- A `WINDOW AGGREGATE` operator
- A Kafka source operator
- A Sink operator

In Flink SQL, the planner translates your SQL into a DAG of operators automatically.

Task

A **Task** is the unit of work that gets scheduled and executed on a TaskManager. It corresponds to a **chain of one or more operators** that Flink has fused together for efficiency.

Flink aggressively **chains** operators that can run in the same thread (same parallelism, no shuffle boundary between them) into a single Task to reduce serialization and threading overhead. So a `Source → Map → Filter` might become one Task rather than three.

Subtask

A **Subtask** is a **parallel instance of a Task**. This is where Flink's parallelism actually lives.

If a Task has parallelism = 4, there are 4 Subtasks, each processing a different partition of the data. Each Subtask:

- Runs in its own thread on a TaskManager slot
- Maintains its own independent state
- Processes its own slice of the input stream

Concrete Example

```
Kafka Source (parallelism=4) → Map (parallelism=4) → KeyBy → Aggregate
(parallelism=4) → Sink (parallelism=2)
```

Concept	Example
Operators	Source, Map, Aggregate, Sink
Tasks	Source+Map chained = 1 Task; Aggregate = 1 Task; Sink = 1 Task
Subtasks	Source+Map Task has 4 subtasks; Sink Task has 2 subtasks

---

Slots and Subtasks

Each **Subtask requires one slot** on a TaskManager. Flink's slot sharing allows subtasks from different Tasks (of the same job) to share a slot, so by default one slot can hold a full **pipeline slice** of your job — one subtask from each Task stage.

Relevance to Confluent Cloud for Apache Flink

When you size a Confluent Cloud for Apache Flink compute pool, the Confluent Flink Unit (CFUs) you allocate determine how many parallel subtasks can run concurrently. If your job has high parallelism (many subtasks), you need enough CFUs to schedule them all — which is why understanding this hierarchy matters when tuning CFU allocation and Autopilot behavior for pipelines.